UTEC
Universidad Tecnológica

# Proyecto

## Aplicación Completa CRUD con Menús e Interacción desde Consola

### 1. Introducción al Proyecto CRUD

**Definición y alcance del proyecto** El objetivo del proyecto es desarrollar una aplicación CRUD (Create, Read, Update, Delete) que permita la gestión de libros y autores. La aplicación debe ser capaz de realizar las operaciones básicas de un CRUD sobre estas dos entidades y gestionar la relación entre ellas. Además, la interacción se realizará desde la consola, brindando un menú interactivo al usuario.

**Requerimientos funcionales y no funcionales**

- **Funcionales**:
  - Crear, leer, actualizar y eliminar registros de libros y autores.
  - Vincular libros con autores, permitiendo la gestión de la relación.
  - Navegar por menús interactivos para realizar las operaciones CRUD.
- **No funcionales**:
  - La aplicación debe ser fácil de usar y mantener.
  - Debe ser modular y seguir buenas prácticas de programación, incluyendo el uso de patrones de diseño.
  - La aplicación debe funcionar correctamente en cualquier entorno compatible con Java.

### 2. Diseño de la Aplicación

**Estructura del proyecto y organización del código**

- El proyecto se estructurará en diferentes paquetes, cada uno de los cuales gestionará un aspecto específico del proyecto:
  - `modelo`: Contendrá las clases `Libro` y `Autor`.
  - `dao`: Implementación de los Data Access Objects (DAOs) para la persistencia de datos.
  - `servicio`: Gestión de la lógica de negocio y la interacción con los DAOs.
  - `consola`: Manejo de la interacción con el usuario y los menús.
  - `util`: Clases de utilidad, incluyendo la conexión a la base de datos.

UTEC
Universidad Tecnológica

**Modelado de datos y diseño de clases**

- **Libro**: Tendrá atributos como `id`, `título`, `año de publicación` y una referencia a un `Autor`.
- **Autor**: Incluirá atributos como `id`, `nombre`, `apellido` y una lista de `Libros`.
- La relación entre `Libro` y `Autor` será de uno a muchos, donde un autor puede escribir varios libros, pero un libro solo tiene un autor.

**3. Implementación de Patrones Creacionales**

**Uso de Singleton para la conexión a la base de datos**

- Se implementará el patrón Singleton para gestionar la conexión a la base de datos, asegurando que solo exista una instancia de conexión durante toda la vida de la aplicación.

**4. Implementación de Patrones Estructurales**

**Uso de Facade para simplificación de interfaces**

- Se implementará un `Facade` que unifique la interacción entre los diferentes servicios y DAOs, proporcionando una interfaz sencilla y coherente para las operaciones CRUD.

**Uso de otros patrones, si aplica, para los DAOs**

- Si es aplicable, se podrán utilizar otros patrones estructurales como el **DAO Pattern** para la separación clara entre la lógica de negocio y la capa de persistencia de datos.

**5. Interacción con el Usuario desde Consola**

**Diseño de menús interactivos**

- Se diseñará un sistema de menús que permita al usuario realizar las operaciones CRUD. El menú debe ser intuitivo, mostrando opciones claras y permitiendo navegar entre las diferentes funcionalidades de la aplicación.

**Gestión de entrada y validación de datos del usuario**

- Se implementarán mecanismos para validar la entrada del usuario, asegurando que los datos ingresados sean correctos antes de procesarlos. Por ejemplo, verificar que un `ID` existe antes de actualizar o eliminar un registro.

**6. Pruebas y Validación**

UTEC
Universidad Tecnológica

**Pruebas unitarias y de integración con JUnit**

- Se desarrollarán pruebas unitarias para cada clase del proyecto, especialmente en las capas de servicio y DAO, utilizando **JUnit** para asegurar que cada componente funcione correctamente de manera aislada.
- También se realizarán pruebas de integración para validar que los diferentes componentes funcionan correctamente cuando se combinan.

**Validación funcional del proyecto completo**

- Finalmente, se validará la funcionalidad completa del proyecto, asegurando que todas las operaciones CRUD, la interacción con el usuario desde la consola y la gestión de la relación entre libros y autores funcionen como se espera.

Este esquema provee una estructura clara para abordar el desarrollo del proyecto, asegurando la aplicación de buenas prácticas y la correcta implementación de patrones de diseño.

## Estructura del Proyecto

```
src/
uy/utec/pa
│
├── consola/
│   └── Main.java
│
├── dao/
│   ├── AutorDAO.java
│   └── LibroDAO.java
│
├── modelo/
│   ├── Autor.java
│   └── Libro.java
│
├── servicio/
│   ├── AutorService.java
│   └── LibroService.java
│
└── util/
    └── DatabaseConnection.java
```

## 1. Clases del Modelo

UTEC
Universidad Tecnológica

**Autor.java**

```java
package uy.utec.pa.modelo;

import java.util.ArrayList;
import java.util.List;

public class Autor {
    private int id;
    private String nombre;
    private String apellido;
    private List<Libro> libros;

    public Autor(int id, String nombre, String apellido) {
        this.id = id;
        this.nombre = nombre;
        this.apellido = apellido;
        this.libros = new ArrayList<>();
    }

    // Getters y Setters
    public int getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    public String getApellido() {
        return apellido;
    }

    public List<Libro> getLibros() {
        return libros;
    }
}
```

**Libro.java**

UTEC
Universidad Tecnológica

```java
package uy.utec.pa.modelo;

public class Libro {
    private int id;
    private String titulo;
    private int anioPublicacion;
    private Autor autor;

    public Libro(int id, String titulo, int anioPublicacion, Autor
autor) {
        this.id = id;
        this.titulo = titulo;
        this.anioPublicacion = anioPublicacion;
        this.autor = autor;
    }

    // Getters y Setters
    public int getId() {
        return id;
    }

    public String getTitulo() {
        return titulo;
    }

    public int getAnioPublicacion() {
        return anioPublicacion;
    }

    public Autor getAutor() {
        return autor;
    }
}
```

## 2. Implementación de DAOs

### DatabaseConnection.java (Singleton)

```java
package uy.utec.pa.util;
```

UTEC
Universidad Tecnológica

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseConnection {
    private static DatabaseConnection instance;
    private Connection connection;

    private DatabaseConnection() {
        try {
            connection =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/tu_base_de
_datos", "usuario", "contraseña");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public static DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }

    public Connection getConnection() {
        return connection;
    }
}
```

**AutorDAO.java**

```
package uy.utec.pa.dao;

import uy.utec.pa.modelo.Autor;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
```

UTEC
Universidad Tecnológica

```
public class AutorDAO {
    public void agregarAutor(Autor autor) {
        Connection connection =
DatabaseConnection.getInstance().getConnection();
        String sql = "INSERT INTO autores (id, nombre, apellido) VALUES
(?, ?, ?)";
        try (PreparedStatement statement =
connection.prepareStatement(sql)) {
            statement.setInt(1, autor.getId());
            statement.setString(2, autor.getNombre());
            statement.setString(3, autor.getApellido());
            statement.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public List<Autor> obtenerAutores() {
        List<Autor> autores = new ArrayList<>();
        Connection connection =
DatabaseConnection.getInstance().getConnection();
        String sql = "SELECT * FROM autores";
        try (Statement statement = connection.createStatement();
             ResultSet resultSet = statement.executeQuery(sql)) {
            while (resultSet.next()) {
                int id = resultSet.getInt("id");
                String nombre = resultSet.getString("nombre");
                String apellido = resultSet.getString("apellido");
                autores.add(new Autor(id, nombre, apellido));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return autores;
    }

    public void actualizarAutor(Autor autor) {
        Connection connection =
DatabaseConnection.getInstance().getConnection();
        String sql = "UPDATE autores SET nombre = ?, apellido = ? WHERE
id = ?";
        try (PreparedStatement statement =
```

UTEC
Universidad Tecnológica

```
connection.prepareStatement(sql)) {
            statement.setString(1, autor.getNombre());
            statement.setString(2, autor.getApellido());
            statement.setInt(3, autor.getId());
            statement.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public void eliminarAutor(int id) {
        Connection connection =
DatabaseConnection.getInstance().getConnection();
        String sql = "DELETE FROM autores WHERE id = ?";
        try (PreparedStatement statement =
connection.prepareStatement(sql)) {
            statement.setInt(1, id);
            statement.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

**LibroDAO.java**

```
package uy.utec.pa.dao;

import uy.utec.pa.modelo.Libro;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class LibroDAO {
    public void agregarLibro(Libro libro) {
        Connection connection =
DatabaseConnection.getInstance().getConnection();
        String sql = "INSERT INTO libros (id, titulo, anio_publicacion,
autor_id) VALUES (?, ?, ?, ?)";
        try (PreparedStatement statement =
```

UTEC
Universidad Tecnológica

```
connection.prepareStatement(sql)) {
            statement.setInt(1, libro.getId());
            statement.setString(2, libro.getTitulo());
            statement.setInt(3, libro.getAnioPublicacion());
            statement.setInt(4, libro.getAutor().getId());
            statement.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public List<Libro> obtenerLibros() {
        List<Libro> libros = new ArrayList<>();
        Connection connection =
DatabaseConnection.getInstance().getConnection();
        String sql = "SELECT l.id, l.titulo, l.anio_publicacion, a.id AS
autor_id, a.nombre, a.apellido " +
                     "FROM libros l JOIN autores a ON l.autor_id =
a.id";
        try (Statement statement = connection.createStatement();
             ResultSet resultSet = statement.executeQuery(sql)) {
            while (resultSet.next()) {
                int id = resultSet.getInt("id");
                String titulo = resultSet.getString("titulo");
                int anioPublicacion =
resultSet.getInt("anio_publicacion");
                int autorId = resultSet.getInt("autor_id");
                String autorNombre = resultSet.getString("nombre");
                String autorApellido = resultSet.getString("apellido");
                Autor autor = new Autor(autorId, autorNombre,
autorApellido);
                libros.add(new Libro(id, titulo, anioPublicacion,
autor));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return libros;
    }

    public void actualizarLibro(Libro libro) {
        Connection connection =
```

```
DatabaseConnection.getInstance().getConnection();
        String sql = "UPDATE libros SET titulo = ?, anio_publicacion =
?, autor_id = ? WHERE id = ?";
        try (PreparedStatement statement =
connection.prepareStatement(sql)) {
            statement.setString(1, libro.getTitulo());
            statement.setInt(2, libro.getAnioPublicacion());
            statement.setInt(3, libro.getAutor().getId());
            statement.setInt(4, libro.getId());
            statement.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    public void eliminarLibro(int id) {
        Connection connection =
DatabaseConnection.getInstance().getConnection();
        String sql = "DELETE FROM libros WHERE id = ?";
        try (PreparedStatement statement =
connection.prepareStatement(sql)) {
            statement.setInt(1, id);
            statement.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

## 3. Servicios

### AutorService.java

```
package uy.utec.pa.servicio;

import uy.utec.pa.dao.AutorDAO;
import uy.utec.pa.modelo.Autor;
import java.util.List;

public class AutorService {
```

```
    private AutorDAO autorDAO;

    public AutorService() {
        this.autorDAO = new AutorDAO();
    }

    public void agregarAutor(Autor autor) {
        autorDAO.agregarAutor(autor);
    }

    public List<Autor> obtenerAutores() {
        return autorDAO.obtenerAutores();
    }

    public void actualizarAutor(Autor autor) {
        autorDAO.actualizarAutor(autor);
    }

    public void eliminarAutor(int id) {
        autorDAO.eliminarAutor(id);
    }
}
```

**LibroService.java**

```
package uy.utec.pa.servicio;

import uy.utec.pa.dao.LibroDAO;
import uy.utec.pa.modelo.Libro;
import java.util.List;

public class LibroService {
    private LibroDAO libroDAO;

    public LibroService() {
        this.libroDAO = new LibroDAO();
    }

    public void agregarLibro(Libro libro) {
        libroDAO.agregarLibro(libro);
```

UTEC
Universidad Tecnológica

```
    }

    public List<Libro> obtenerLibros() {
        return libroDAO.obtenerLibros();
    }

    public void actualizarLibro(Libro libro) {
        libroDAO.actualizarLibro(libro);
    }

    public void eliminarLibro(int id) {
        libroDAO.eliminarLibro(id);
    }
}
```

## 4. Interacción con el Usuario

### Main.java

```
package uy.utec.pa.consola;

import uy.utec.pa.modelo.Autor;
import uy.utec.pa.modelo.Libro;
import uy.utec.pa.servicio.AutorService;
import uy.utec.pa.servicio.LibroService;

import java.util.List;
import java.util.Scanner;

public class Main {
    private static final AutorService autorService = new AutorService();
    private static final LibroService libroService = new LibroService();
    private static final Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        int opcion;
        do {
            System.out.println("Menú:");
            System.out.println("1. Agregar Autor");
            System.out.println("2. Listar Autores");
```

UTEC
Universidad Tecnológica

```
            System.out.println("3. Actualizar Autor");
            System.out.println("4. Eliminar Autor");
            System.out.println("5. Agregar Libro");
            System.out.println("6. Listar Libros");
            System.out.println("7. Actualizar Libro");
            System.out.println("8. Eliminar Libro");
            System.out.println("0. Salir");
            System.out.print("Seleccione una opción: ");
            opcion = scanner.nextInt();
            scanner.nextLine(); // Limpiar el buffer

            switch (opcion) {
                case 1:
                    agregarAutor();
                    break;
                case 2:
                    listarAutores();
                    break;
                case 3:
                    actualizarAutor();
                    break;
                case 4:
                    eliminarAutor();
                    break;
                case 5:
                    agregarLibro();
                    break;
                case 6:
                    listarLibros();
                    break;
                case 7:
                    actualizarLibro();
                    break;
                case 8:
                    eliminarLibro();
                    break;
                case 0:
                    System.out.println("Saliendo...");
                    break;
                default:
                    System.out.println("Opción no válida. Intente de
nuevo.");
```

UTEC
Universidad Tecnológica

```
            }
        } while (opcion != 0);
    }

    private static void agregarAutor() {
        System.out.print("Ingrese ID del autor: ");
        int id = scanner.nextInt();
        scanner.nextLine();
        System.out.print("Ingrese nombre del autor: ");
        String nombre = scanner.nextLine();
        System.out.print("Ingrese apellido del autor: ");
        String apellido = scanner.nextLine();
        autorService.agregarAutor(new Autor(id, nombre, apellido));
        System.out.println("Autor agregado.");
    }

    private static void listarAutores() {
        List<Autor> autores = autorService.obtenerAutores();
        for (Autor autor : autores) {
            System.out.println("ID: " + autor.getId() + ", Nombre: " +
autor.getNombre() + ", Apellido: " + autor.getApellido());
        }
    }

    private static void actualizarAutor() {
        System.out.print("Ingrese ID del autor a actualizar: ");
        int id = scanner.nextInt();
        scanner.nextLine();
        System.out.print("Ingrese nuevo nombre del autor: ");
        String nombre = scanner.nextLine();
        System.out.print("Ingrese nuevo apellido del autor: ");
        String apellido = scanner.nextLine();
        autorService.actualizarAutor(new Autor(id, nombre, apellido));
        System.out.println("Autor actualizado.");
    }

    private static void eliminarAutor() {
        System.out.print("Ingrese ID del autor a eliminar: ");
        int id = scanner.nextInt();
        autorService.eliminarAutor(id);
        System.out.println("Autor eliminado.");
    }
```

UTEC
Universidad Tecnológica

```
    private static void agregarLibro() {
        System.out.print("Ingrese ID del libro: ");
        int id = scanner.nextInt();
        scanner.nextLine();
        System.out.print("Ingrese título del libro: ");
        String titulo = scanner.nextLine();
        System.out.print("Ingrese año de publicación: ");
        int anioPublicacion = scanner.nextInt();
        System.out.print("Ingrese ID del autor: ");
        int autorId = scanner.nextInt();
        // Supongamos que ya tenemos el autor cargado en la base de
datos
        Autor autor = new Autor(autorId, "NombreAutor",
"ApellidoAutor"); // Debe buscar el autor por ID
        libroService.agregarLibro(new Libro(id, titulo, anioPublicacion,
autor));
        System.out.println("Libro agregado.");
    }

    private static void listarLibros() {
        List<Libro> libros = libroService.obtenerLibros();
        for (Libro libro : libros) {
            System.out.println("ID: " + libro.getId() + ", Título: " +
libro.getTitulo() + ", Año: " + libro.getAnioPublicacion() + ", Autor: "
+ libro.getAutor().getNombre() + " " + libro.getAutor().getApellido());
        }
    }

    private static void actualizarLibro() {
        System.out.print("Ingrese ID del libro a actualizar: ");
        int id = scanner.nextInt();
        scanner.nextLine();
        System.out.print("Ingrese nuevo título del libro: ");
        String titulo = scanner.nextLine();
        System.out.print("Ingrese nuevo año de publicación: ");
        int anioPublicacion = scanner.nextInt();
        System.out.print("Ingrese nuevo ID del autor: ");
        int autorId = scanner.nextInt();
        // Debe buscar el autor por ID
        Autor autor = new Autor(autorId, "NombreAutor",
"ApellidoAutor"); // Debe buscar el autor por ID
```

UTEC
Universidad Tecnológica

```
        libroService.actualizarLibro(new Libro(id, titulo,
anioPublicacion, autor));
        System.out.println("Libro actualizado.");
    }

    private static void eliminarLibro() {
        System.out.print("Ingrese ID del libro a eliminar: ");
        int id = scanner.nextInt();
        libroService.eliminarLibro(id);
        System.out.println("Libro eliminado.");
    }
}
```

## 5. Pruebas y Validación

Para escribir los **tests unitarios** para los DAOs, necesitamos simular su comportamiento sin depender directamente de una base de datos. Esto se puede hacer utilizando **mocks** para aislar los DAOs y probar su lógica. Vamos a usar **JUnit** para los tests y **Mockito** para crear estos mocks.

Para realizar pruebas unitarias del DAO `AutorDAO`, necesitamos hacer uso de un enfoque como **Mockito** para simular (mockear) las dependencias, como la conexión a la base de datos, dado que no queremos hacer pruebas directamente sobre una base de datos real. El test se enfocará en verificar que los métodos interactúan correctamente con la base de datos, sin realizar consultas reales.

A continuación te doy un ejemplo de cómo escribir un test unitario utilizando **JUnit 5** y **Mockito** para el DAO `AutorDAO` que has proporcionado.

### Dependencias necesarias en Maven

Añade las siguientes dependencias para **JUnit** y **Mockito** en tu archivo `pom.xml`:

```
<dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-engine</artifactId>
      <version>5.10.2</version>
      <scope>test</scope>
    </dependency>
    <dependency>
```

UTEC
Universidad Tecnológica

```
      <groupId>org.mockito</groupId>
      <artifactId>mockito-core</artifactId>
      <version>5.11.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
```

## Test unitario para `AutorDAO`

A continuación, te muestro cómo se puede probar el DAO usando **Mockito** para simular la conexión a la base de datos y las consultas:

```
package uy.utec.pa;
import org.junit.jupiter.api.*;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;
import uy.utec.pa.modelo.Autor;
import uy.utec.pa.dao.AutorDAO;
import uy.utec.pa.util.DatabaseConnection;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

class AutorDAOTest {

    @Mock
    private Connection connection;

    @Mock
    private PreparedStatement preparedStatement;

    @Mock
    private Statement statement;
```

UTEC
Universidad Tecnológica

```
    @Mock
    private ResultSet resultSet;

    @InjectMocks
    private AutorDAO autorDAO;

    @BeforeEach
    void setUp() throws SQLException {
        // Inicializar los mocks de Mockito
        MockitoAnnotations.openMocks(this);

        // Mockear el Singleton para devolver la conexión mockeada
        DatabaseConnection singletonMock =
mock(DatabaseConnection.class);
        when(singletonMock.getConnection()).thenReturn(connection);

        // Reemplazar la instancia del Singleton por el mock
        mockStatic(DatabaseConnection.class);

when(DatabaseConnection.getInstance()).thenReturn(singletonMock);

        // Simular que el Connection retorna el PreparedStatement y
Statement correctos

when(connection.prepareStatement(anyString())).thenReturn(preparedStatem
ent);
        when(connection.createStatement()).thenReturn(statement);
    }

    @AfterEach
    void setUpAll(){
        clearAllCaches();
    }

    @Test
    void testAgregarAutor() throws SQLException {
        Autor autor = new Autor(1, "Juan", "Perez");

        // Simular la ejecución de update
        when(preparedStatement.executeUpdate()).thenReturn(1);
```

UTEC
Universidad Tecnológica

```
        // Llamar al método agregarAutor
        autorDAO.agregarAutor(autor);

        // Verificar que el PreparedStatement fue llamado con los
parámetros correctos
        verify(preparedStatement).setInt(1, autor.getId());
        verify(preparedStatement).setString(2, autor.getNombre());
        verify(preparedStatement).setString(3, autor.getApellido());
        verify(preparedStatement).executeUpdate();
    }

    @Test
    void testObtenerAutores() throws SQLException {
        List<Autor> expectedAutores = new ArrayList<>();
        expectedAutores.add(new Autor(1, "Juan", "Perez"));
        expectedAutores.add(new Autor(2, "Ana", "Gomez"));

        // Simular el ResultSet con datos ficticios
        when(resultSet.next()).thenReturn(true, true, false); // Dos
registros
        when(resultSet.getInt("id")).thenReturn(1, 2);
        when(resultSet.getString("nombre")).thenReturn("Juan", "Ana");
        when(resultSet.getString("apellido")).thenReturn("Perez",
"Gomez");

        // Simular la ejecución del query
        when(statement.executeQuery(anyString())).thenReturn(resultSet);

        // Llamar al método obtenerAutores
        List<Autor> autores = autorDAO.obtenerAutores();

        // Verificar que se devolvieron los autores correctos
        assertEquals(expectedAutores.size(), autores.size());
        assertEquals(expectedAutores.get(0).getNombre(),
autores.get(0).getNombre());
        assertEquals(expectedAutores.get(1).getNombre(),
autores.get(1).getNombre());
    }

    @Test
    void testActualizarAutor() throws SQLException {
        Autor autor = new Autor(1, "Juan", "Lopez");
```

UTEC
Universidad Tecnológica

```
        // Simular la ejecución de update
        when(preparedStatement.executeUpdate()).thenReturn(1);

        // Llamar al método actualizarAutor
        autorDAO.actualizarAutor(autor);

        // Verificar que el PreparedStatement fue llamado con los
parámetros correctos
        verify(preparedStatement).setString(1, autor.getNombre());
        verify(preparedStatement).setString(2, autor.getApellido());
        verify(preparedStatement).setInt(3, autor.getId());
        verify(preparedStatement).executeUpdate();
    }

    @Test
    void testEliminarAutor() throws SQLException {
        int autorId = 1;

        // Simular la ejecución de delete
        when(preparedStatement.executeUpdate()).thenReturn(1);

        // Llamar al método eliminarAutor
        autorDAO.eliminarAutor(autorId);

        // Verificar que el PreparedStatement fue llamado con los
parámetros correctos
        verify(preparedStatement).setInt(1, autorId);
        verify(preparedStatement).executeUpdate();
    }
}
```

## Explicación del código:

Este código es un conjunto de pruebas unitarias para la clase `AutorDAO`, que realiza operaciones de acceso a datos sobre una base de datos. Utiliza `JUnit 5` para definir y ejecutar las pruebas, y `Mockito` para simular (mockear) objetos como `Connection`, `PreparedStatement`, `Statement` y `ResultSet`, evitando así la necesidad de conectarse a una base de datos real.

UTEC
Universidad Tecnológica

## Imports

```
import org.junit.jupiter.api.*;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;
import uy.utec.pa.modelo.Autor;
import uy.utec.pa.dao.AutorDAO;
import uy.utec.pa.util.DatabaseConnection;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;
```

- Se importan clases y métodos necesarios para realizar las pruebas con `JUnit` y `Mockito`.
- `@Mock` y `@InjectMocks` se utilizan para simular y crear dependencias.
- Se utiliza `mockStatic` para simular la clase singleton `DatabaseConnection`, permitiendo que devuelva un `Connection` simulado.

## Variables Simuladas (Mock)

```
@Mock
private Connection connection;

@Mock
private PreparedStatement preparedStatement;

@Mock
private Statement statement;

@Mock
private ResultSet resultSet;

@InjectMocks
private AutorDAO autorDAO;
```

- Se definen los objetos de base de datos simulados: `Connection`, `PreparedStatement`, `Statement` y `ResultSet`. Estos objetos no son reales, sino simulaciones que nos permiten controlar su comportamiento en las pruebas.
- `@InjectMocks` inyecta las dependencias simuladas en `autorDAO`, lo que permite probar su comportamiento sin acceder a una base de datos real.

### Configuración del Entorno de Prueba (`@BeforeEach`)

```
@BeforeEach
void setUp() throws SQLException {
    // Inicializar los mocks de Mockito
    MockitoAnnotations.openMocks(this);

    // Mockear el Singleton para devolver la conexión mockeada
    DatabaseConnection singletonMock = mock(DatabaseConnection.class);
    when(singletonMock.getConnection()).thenReturn(connection);

    // Reemplazar la instancia del Singleton por el mock
    mockStatic(DatabaseConnection.class);
    when(DatabaseConnection.getInstance()).thenReturn(singletonMock);

    // Simular que el Connection retorna el PreparedStatement y
Statement correctos

when(connection.prepareStatement(anyString())).thenReturn(preparedStatem
ent);
    when(connection.createStatement()).thenReturn(statement);
}
```

- **`@BeforeEach`**: Este método se ejecuta antes de cada prueba para inicializar los objetos simulados.
- Se mockea la clase singleton `DatabaseConnection` para que devuelva una conexión simulada. Esto asegura que cuando se llame a `getConnection()`, no se esté conectando a una base de datos real.
- Se especifica qué debe retornar la conexión simulada cuando se invoquen métodos como `prepareStatement()` y `createStatement()`.

### Limpieza después de las pruebas (`@AfterEach`)

UTEC
Universidad Tecnológica

```
@AfterEach
void setUpAll(){
    clearAllCaches();
}
```

- Limpia los caches de los mocks estáticos para evitar interferencias entre las pruebas.

### Prueba de agregar un autor (`testAgregarAutor`)

```
@Test
void testAgregarAutor() throws SQLException {
    Autor autor = new Autor(1, "Juan", "Perez");

    // Simular la ejecución de update
    when(preparedStatement.executeUpdate()).thenReturn(1);

    // Llamar al método agregarAutor
    autorDAO.agregarAutor(autor);

    // Verificar que el PreparedStatement fue llamado con los parámetros
correctos
    verify(preparedStatement).setInt(1, autor.getId());
    verify(preparedStatement).setString(2, autor.getNombre());
    verify(preparedStatement).setString(3, autor.getApellido());
    verify(preparedStatement).executeUpdate();
}
```

- Se crea un objeto `Autor` con los datos que se van a utilizar en la prueba.
- Se simula que la llamada a `executeUpdate()` retorna 1, lo cual indica que la operación fue exitosa.
- Se verifica que el método `setInt`, `setString`, y `executeUpdate` del `PreparedStatement` hayan sido invocados con los parámetros correctos.

### Prueba de obtener autores (`testObtenerAutores`)

```
@Test
void testObtenerAutores() throws SQLException {
    List<Autor> expectedAutores = new ArrayList<>();
    expectedAutores.add(new Autor(1, "Juan", "Perez"));
```

UTEC
Universidad Tecnológica

```
    expectedAutores.add(new Autor(2, "Ana", "Gomez"));

    // Simular el ResultSet con datos ficticios
    when(resultSet.next()).thenReturn(true, true, false); // Dos
registros
    when(resultSet.getInt("id")).thenReturn(1, 2);
    when(resultSet.getString("nombre")).thenReturn("Juan", "Ana");
    when(resultSet.getString("apellido")).thenReturn("Perez", "Gomez");

    // Simular la ejecución del query
    when(statement.executeQuery(anyString())).thenReturn(resultSet);

    // Llamar al método obtenerAutores
    List<Autor> autores = autorDAO.obtenerAutores();

    // Verificar que se devolvieron los autores correctos
    assertEquals(expectedAutores.size(), autores.size());
    assertEquals(expectedAutores.get(0).getNombre(),
autores.get(0).getNombre());
    assertEquals(expectedAutores.get(1).getNombre(),
autores.get(1).getNombre());
}
```

- Se simula un `ResultSet` que contiene dos registros.
- Se especifica que el `ResultSet` debe devolver los valores correspondientes cuando se llamen `getInt` y `getString`.
- Se verifica que los autores obtenidos a través del método `obtenerAutores` coincidan con los autores esperados.

### Pruebas de actualizar y eliminar autores (`testActualizarAutor` y `testEliminarAutor`)

```
@Test
void testActualizarAutor() throws SQLException {
    Autor autor = new Autor(1, "Juan", "Lopez");

    // Simular la ejecución de update
    when(preparedStatement.executeUpdate()).thenReturn(1);

    // Llamar al método actualizarAutor
```

UTEC
Universidad Tecnológica

```
    autorDAO.actualizarAutor(autor);

    // Verificar que el PreparedStatement fue llamado con los parámetros
correctos
    verify(preparedStatement).setString(1, autor.getNombre());
    verify(preparedStatement).setString(2, autor.getApellido());
    verify(preparedStatement).setInt(3, autor.getId());
    verify(preparedStatement).executeUpdate();
}

@Test
void testEliminarAutor() throws SQLException {
    int autorId = 1;

    // Simular la ejecución de delete
    when(preparedStatement.executeUpdate()).thenReturn(1);

    // Llamar al método eliminarAutor
    autorDAO.eliminarAutor(autorId);

    // Verificar que el PreparedStatement fue llamado con los parámetros
correctos
    verify(preparedStatement).setInt(1, autorId);
    verify(preparedStatement).executeUpdate();
}
```

- En ambas pruebas se verifica que los métodos `actualizarAutor` y `eliminarAutor` llamen al `PreparedStatement` con los valores correctos y que la operación haya sido ejecutada correctamente.

## Importante: Mocks en pruebas unitarias

En una prueba unitaria **no se prueba contra terceros** ni contra una **base de datos real**. El objetivo es aislar el código que se está probando, de manera que sólo se evalúe el comportamiento de la lógica interna. En este caso, se simula (mockea) la base de datos y las conexiones para que no sea necesario interactuar con un sistema externo durante la prueba. Esto ayuda a que las pruebas sean rápidas, predecibles y que sólo verifiquen la lógica de la clase `AutorDAO`.

UTEC
Universidad Tecnológica

## Test unitario para `LibroDAO`

Ahora vamos a realizar los tests unitarios para los métodos de la clase `LibroDAO` usando **Mockito**:

```
package uy.utec.pa;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;
import uy.utec.pa.dao.LibroDAO;
import uy.utec.pa.modelo.Autor;
import uy.utec.pa.modelo.Libro;
import uy.utec.pa.util.DatabaseConnection;

import java.sql.*;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

class LibroDAOTest {

    @Mock
    private Connection connection;

    @Mock
    private PreparedStatement preparedStatement;

    @Mock
    private Statement statement;

    @Mock
    private ResultSet resultSet;

    @InjectMocks
    private LibroDAO libroDAO;
```

UTEC
Universidad Tecnológica

```
    @BeforeEach
    void setUp() throws SQLException {
        // Inicializar los mocks de Mockito
        MockitoAnnotations.openMocks(this);

        // Mockear el Singleton para devolver la conexión mockeada
        DatabaseConnection singletonMock =
mock(DatabaseConnection.class);
        when(singletonMock.getConnection()).thenReturn(connection);

        // Reemplazar la instancia del Singleton por el mock
        mockStatic(DatabaseConnection.class);

when(DatabaseConnection.getInstance()).thenReturn(singletonMock);

        // Simular que el Connection retorna el PreparedStatement y
Statement correctos

when(connection.prepareStatement(anyString())).thenReturn(preparedStatem
ent);
        when(connection.createStatement()).thenReturn(statement);
    }

    @AfterEach
    void setUpAll(){
        clearAllCaches();
    }

    @Test
    void testAgregarLibro() throws Exception {
        Autor autor = new Autor(1, "Juan", "Perez");
        Libro libro = new Libro(1, "El libro", 2021, autor);

        // Simular la ejecución de update
        when(preparedStatement.executeUpdate()).thenReturn(1);

        // Llamar al método agregarLibro
        libroDAO.agregarLibro(libro);

        // Verificar que el PreparedStatement fue llamado con los
parámetros correctos
```

UTEC
Universidad Tecnológica

```
        verify(preparedStatement).setInt(1, libro.getId());
        verify(preparedStatement).setString(2, libro.getTitulo());
        verify(preparedStatement).setInt(3, libro.getAnioPublicacion());
        verify(preparedStatement).setInt(4, libro.getAutor().getId());
        verify(preparedStatement).executeUpdate();
    }

    @Test
    void testObtenerLibros() throws Exception {
        // Simular el ResultSet con datos ficticios
        when(resultSet.next()).thenReturn(true).thenReturn(false); // Un
solo registro
        when(resultSet.getInt("id")).thenReturn(1);
        when(resultSet.getString("titulo")).thenReturn("El libro");
        when(resultSet.getInt("anio_publicacion")).thenReturn(2021);
        when(resultSet.getInt("autor_id")).thenReturn(1);
        when(resultSet.getString("nombre")).thenReturn("Juan");
        when(resultSet.getString("apellido")).thenReturn("Perez");
        when(statement.executeQuery(anyString())).thenReturn(resultSet);

        // Llamar al método obtenerLibros
        List<Libro> libros = libroDAO.obtenerLibros();

        // Verificar que se obtuvo el libro correcto
        assertEquals(1, libros.size());
        assertEquals(1, libros.get(0).getId());
        assertEquals("El libro", libros.get(0).getTitulo());
        assertEquals(2021, libros.get(0).getAnioPublicacion());
        assertEquals(1, libros.get(0).getAutor().getId());
        assertEquals("Juan", libros.get(0).getAutor().getNombre());
        assertEquals("Perez", libros.get(0).getAutor().getApellido());
    }

    @Test
    void testActualizarLibro() throws Exception {
        Autor autor = new Autor(1, "Juan", "Gonzalez");
        Libro libro = new Libro(1, "El libro actualizado", 2022, autor);

        // Simular la ejecución de update
        when(preparedStatement.executeUpdate()).thenReturn(1);

        // Llamar al método actualizarLibro
```

UTEC
Universidad Tecnológica

```
        libroDAO.actualizarLibro(libro);

        // Verificar que el PreparedStatement fue llamado con los
parámetros correctos
        verify(preparedStatement).setString(1, libro.getTitulo());
        verify(preparedStatement).setInt(2, libro.getAnioPublicacion());
        verify(preparedStatement).setInt(3, libro.getAutor().getId());
        verify(preparedStatement).setInt(4, libro.getId());
        verify(preparedStatement).executeUpdate();
    }

    @Test
    void testEliminarLibro() throws Exception {
        int libroId = 1;

        // Simular la ejecución de update
        when(preparedStatement.executeUpdate()).thenReturn(1);

        // Llamar al método eliminarLibro
        libroDAO.eliminarLibro(libroId);

        // Verificar que el PreparedStatement fue llamado con el
parámetro correcto
        verify(preparedStatement).setInt(1, libroId);
        verify(preparedStatement).executeUpdate();
    }
}
```

## Agregando más validaciones

Supongamos que queremos modificar el método guardar del **Autor** para que valide que el nombre no sea vacío y, si lo es, lance una excepción.

## Implementación del método en `AutorDAO`

El método `guardar()` en `AutorDAO` validará que el nombre del autor no esté vacío antes de proceder con la operación de guardado. Si el nombre es vacío, lanzará una excepción `IllegalArgumentException`.

**AutorDao.java**

```
public class AutorDAO {

    public boolean guardar(Autor autor) {
        if (autor.getNombre() == null ||
autor.getNombre().trim().isEmpty()) {
            throw new IllegalArgumentException("El nombre del autor no
puede estar vacío.");
        }
        // Simulación de guardar en DB
        return true;
    }

}
```

En el método `guardar()`, se verifica que el nombre del autor no sea `null` ni esté vacío o compuesto únicamente por espacios en blanco. Si no cumple estas condiciones, lanza una excepción `IllegalArgumentException`.

## Implementación del test unitario para `guardar()`

Ahora vamos a crear el test unitario que validará este comportamiento, es decir, que se lance la excepción cuando se intenta guardar un autor con un nombre vacío o `null`.

**AutorDAOTest.java**

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

class AutorDAOTest {

    private AutorDao autorDao;
```

UTEC
Universidad Tecnológica

```
    @BeforeEach
    void setUp() {
        autorDao = new AutorDao(); // No necesitamos un mock aquí porque
queremos probar la lógica real
    }

    @Test
    void testGuardarNombreVacio() {
        Autor autorConNombreVacio = new Autor(1, ""); // Nombre vacío

        // Validamos que lanzar la excepción cuando el nombre está vacío
        Exception exception =
assertThrows(IllegalArgumentException.class, () -> {
            autorDao.guardar(autorConNombreVacio);
        });

        String expectedMessage = "El nombre del autor no puede estar
vacío.";
        String actualMessage = exception.getMessage();

        assertTrue(actualMessage.contains(expectedMessage));
    }

    @Test
    void testGuardarNombreNull() {
        Autor autorConNombreNull = new Autor(2, null); // Nombre null

        // Validamos que lanzar la excepción cuando el nombre es null
        Exception exception =
assertThrows(IllegalArgumentException.class, () -> {
            autorDao.guardar(autorConNombreNull);
        });

        String expectedMessage = "El nombre del autor no puede estar
vacío.";
        String actualMessage = exception.getMessage();

        assertTrue(actualMessage.contains(expectedMessage));
    }

    @Test
```

UTEC
Universidad Tecnológica

```
    void testGuardarNombreValido() {
        Autor autorValido = new Autor(3, "Autor Valido");

        // Validamos que se puede guardar correctamente cuando el nombre
es válido
        boolean resultado = autorDao.guardar(autorValido);

        assertTrue(resultado);
    }
}
```

## Explicación del test

- **`assertThrows()`**: Este método se utiliza para comprobar que una excepción es lanzada. En este caso, verificamos que al intentar guardar un autor con un nombre vacío o `null`, se lance una excepción de tipo `IllegalArgumentException`.
- **`assertTrue()`**: Se usa para validar que el mensaje de la excepción contiene el texto esperado: "El nombre del autor no puede estar vacío."
- **Tests de casos válidos**: Además de probar los casos inválidos (nombre vacío o `null`), también se incluye un test para un caso válido donde el nombre del autor es correcto y se espera que el método `guardar()` retorne `true`.

Repo Git: https://git.utec.edu.uy/pda-2024/m5/proyecto